



# MIAE 215 Arduino FINAL EXAM Cheat Sheet: Data Types & Functions

Premium

Course: Mechanical, Industrial & Aerospace engineering (MIAE 215) 378 documents University: Concordia University Chase Pezzi 2025/2026

Download

1 0 Save Share Info

This cheat sheet provides a comprehensive overview of Arduino programming concepts, including data types, control flow, func...more

MIAE 215 — C++ & Arduino FINAL EXAM CHEAT SHEET

DATA TYPES & DECLARATIONS

```
int i=0; float f=3.14; double d;
bool b=true; char ch='A'; long L=0;
const int MAX=100; unsigned int u=5;
Enum: enum days {Mon=1,Tue,Wed,Thu,Fri,Sat,Sun};
days today = Sat; // Mon=1, Tue=2 ...
auto-inc
Struct: groups mixed types; access fields with .
struct Student { int id; float gpa; };
Student s; s.id=123; s.gpa=3.9;
Nested struct access: today.time_now.hour = 7;
■ No == on structs — compare field by field.
```

OPERATORS

**Arithmetic:** + - \* / % (modulus)  
**Compound:** += -= \*= /= %=  
**Increment:** i++ (use then inc) | ++i (inc then use)  
**Relational:** == != < > <= >=  
**Boolean:** && (AND) || (OR) ! (NOT)  
**Ternary:** (cond) ? val\_true : val\_false  
**Scope resolution:** ::x — accesses global x when local hides it

I/O #include <iostream> #include <iomanip>

```
cout <<< "val=" <<< x <<< endl;
cin >>> a >>> b;
cout <<< setprecision(4) <<< fixed
<<< x;
```

CONTROL FLOW

```
if (x>0) { ... }
else if (x<0) { ... }
else { ... }

switch(x) { case 1: ...; break; default:
...; }
while (cond) { ... } // check before enter
do { ... } while (cond); // runs at least
once
for (int i=0; i<N; i++) { ... }
break → exits loop/switch early. continue → skip to next
iteration.
```

FUNCTIONS

```
int max(int a, int b); // prototype
int max(int a, int b) { // definition
return (a>b) ? a : b;
}
Call-by-Value: copy passed; original unchanged.
Call-by-Reference (&): original IS modified.
void inc(int&a; a) { a++; } // caller's var
changes
Overloading: same name, different param types —
compiler picks best match.
Static local: static int cnt=0; → value persists across calls.
Inline: inline int sq(int x){return x*x;} — no call overhead.
Scope: local hides global; use :: to reach global.
Short-circuit: in A&&B, if A is
```

ARRAYS

```
int a[5] = {10,20,30,40,50}; // 0-indexed
int* p = a; // same as &a[0] (name =
address)
Pass to function — always by reference, changes
propagate:
void fill(int arr[], int size) { ... }
2-D array:
int m[3][4]; m[i][j] = val;
void f(int m[][4], int rows); // must give
cols
Linear search: scan each element, O(n). Works on
unsorted arrays.
Binary search: sorted only; halve search space each step,
O(log n).
Bubble sort: repeated passes, swap adjacent out-of-order
pairs.
Selection sort: find minimum, swap to front, repeat for
remaining.
```

POINTERS

```
int x=5;
int* p = &x; // p stores address of x
*p = 10; // dereference: x is now 10
p++; // moves forward sizeof(int) bytes
void*: generic pointer — must cast before dereferencing:
void* vp = &x;
*(int*)vp = 99;
Struct pointer: use -> instead of (*ptr).field :
Student* ps = &s;
ps->id = 456; // same as (*ps).id = 456
Dynamic memory (heap):
int* arr = new int[N]; // allocate
delete[] arr; // always free!
Pointer arithmetic: ptr+1 moves by sizeof(type) bytes, not 1.
```

CHAR ARRAYS (C-strings) #include <cstring>

```
char s[20] = "hello"; // null-terminated
(\0)
strlen(s) // length (excl \0)
strcpy(dst, src) // copy
strcat(dst, src) // append
strcmp(a, b) // 0 if equal
```

MATH / RANDOM #include <cmath> <cstdlib>

```
sqrt(x) pow(x,y) fabs(x) ceil(x) floor(x)
srand((unsigned)time(0)); // seed with time
rand(); // 0 .. RAND_MAX
```

STEPWISE REFINEMENT

Break problem into sub-problems (top-down design). Each function does ONE thing. Write prototype first, implement later. Aids readability and team development.

Mock exam Summary Quiz

Ask a question about this document



**Inline:** inline int sq(int x){return x\*x;} — no call overhead.  
**Scope:** local hides global; use :: to reach global.  
Short-circuit: in A&&B, if A is false → B is NOT evaluated.

Page 1 | Covers: notes0-8 (per exam guide), notes11, arduino\_notes

## MIAE 215 — C++ &amp; Arduino FINAL EXAM CHEAT SHEET

## FILE I/O #include &lt;fstream&gt;

```
// Constructor style:
ifstream in("in.dat"); ofstream
out("out.dat");
while (in &gt;&gt; x) out &lt;&lt; x
&lt;&lt; endl;
```

```
// Method style:
ifstream in; in.open("f1.dat");
in.close(); in.open("f2.dat"); // reuse
```

```
// Append mode:
ofstream out("log.dat", ios_base::out |
ios_base::app);
```

```
// Binary write:
ofstream out("b.dat", ios::binary |
ios::out);
out.write((char*)&x, sizeof(double));
```

```
// Binary read:
ifstream in("b.dat", ios::binary | ios::in);
in.read((char*)&x, sizeof(double));
while (!in.eof()) {
cout &lt;&lt; x;
in.read((char*)&x, sizeof(double));
}
```

■ Binary: platform-specific — create & read on same OS/hardware.

Precision: #include <iomanip> → out << setprecision(16) << x;

## ARDUINO STRUCTURE &amp; MEMORY

```
void setup() {
Serial.begin(9600); // USB serial
pinMode(pin, OUTPUT); // or INPUT
}
void loop() { /* runs FOREVER */ }
FLASH 32KB (program, non-volatile) | SRAM 2KB
(variables) | EEPROM 1KB
```

## DIGITAL / TIMING / ANALOG

```
pinMode(pin, OUTPUT); digitalWrite(pin,
HIGH);
int v = digitalRead(pin); // HIGH or LOW
```

```
delay(ms); delayMicroseconds(us);
unsigned long t = micros(); // us since start
unsigned long t = millis(); // ms since start
```

```
int v = analogRead(A0); // 0-1023 (A0-A5)
analogWrite(pin, val); // PWM 0-255 (pins
3,5,6,9,10,11)
PWM duty = val/255. ■ Do NOT use analogWrite() for
servos.
```

## SERIAL MONITOR

```
Serial.begin(9600); // in setup()
Serial.print(x); // no newline
Serial.println(x); // with newline
Baud rates: 300 600 1200 2400 4800 9600 19200 57600
115200
```

## ULTRASONIC SENSOR (HC-SR04)

TRIG → 10µs HIGH pulse fires burst. ECHO → HIGH duration ∝ distance.

```
// setup(): pinMode(trig,OUTPUT);
pinMode(echo,INPUT);
digitalWrite(trig,LOW); delay(2);
digitalWrite(trig,HIGH);
delayMicroseconds(11);
digitalWrite(trig,LOW);
while(digitalRead(echo)!=HIGH); // wait HIGH
start=micros();
while(digitalRead(echo)!=LOW); // wait LOW
end=micros();
duration=end-start; // microseconds
distance=duration/58; // centimetres
Formula: distance(cm) = duration(µs)/58
[sound=0.0343cm/µs, /2 round-trip]
```

## SERVO #include &lt;Servo.h&gt;

Servo motor;  
motor.attach(9); // PWM pin  
motor.write(90); // 0 to 180 degrees  
PWM pins: 3 5 6 9 10 11. ■ Use Servo library, NOT analogWrite().

## ARDUINO DATA TYPES

boolean 1B true/false  
byte 1B 0..255  
int 2B -32768..32767  
long 4B +/-2,147,483,648  
float 4B IEEE754 (no double!)  
Use **unsigned long** for micros() / millis() return values.

## KEY REMINDERS

- Arrays pass by reference — no & needed; changes propagate.
- Structs: no == — compare field by field.
- By-value: copy changed. By-reference (&): original changed.
- static local: retains value between calls.
- Short-circuit: A&&B — B skipped if A is false.
- Binary search: array must be sorted.
- delete[] ptr — always free after new[].
- Binary files: same OS/hardware to create & read.
- :: accesses global when local name shadows it.